1. Project Planning & Management

Project: Software-Defined Industrial Visual Anomaly Detection System: Cloud MLOps Pipeline & Simulation
Supervisor: Randa Hesiam
Project start: Sunday 18 October 2026
Final delivery: Saturday 28 November 2026 (6 weeks)

1.1 Project Proposal

Overview

Manufacturers of ceramic tiles, sanitaryware and machined metal parts still rely largely on manual visual inspection at the end of the line. Human inspectors tire, vary in judgement and cannot sustain modern throughput. Hairline cracks, scratches, chips, pits and inclusions are small, low-contrast and often reflective, so defective units regularly reach packaging.
This project designs and implements a computer vision system that inspects every image, localises defects with bounding boxes and confidence scores, and returns a PASS/FAIL verdict in under one second (warm path). The production line is simulated: a Python Camera & Line Simulator streams image events to Amazon S3, which triggers an event-driven AWS serverless pipeline. Results appear in a live web dashboard. No physical hardware or factory access is required.
Flow: Simulator -> S3 -> Lambda -> SageMaker Serverless (YOLOv8 ONNX) -> DynamoDB -> Streamlit dashboard (reads DynamoDB, S3 presigned URLs, CloudWatch).

Objectives

- Detect and localise surface defects on two material families (ceramics and metals) with high recall.
- Return bounding boxes, class label and confidence per finding, plus an overall PASS/FAIL decision.
- Achieve end-to-end warm latency below 1 second per image using ONNX Runtime; report cold-start latency separately.
- Build a fully serverless, event-driven AWS pipeline with traceability of every decision.
- Provide a Python simulator reproducing realistic line behaviour (rate, batches, bursts, defect mix).
- Deliver an interactive web dashboard for monitoring and for the graduation defense.
- Establish a repeatable MLOps loop: drift detection, retraining, quality gate, registry, gated deployment with rollback.

Scope

In scope

- Two material families: ceramics and metals.
- Datasets: MVTec AD, NEU, DAGM, plus one public Kaggle/Roboflow ceramics set; unified classes crack, scratch, chip, pit, stain/rust (final list fixed in M1).
- Production model: YOLOv8 + ONNX Runtime. PatchCore is an offline baseline only and is never deployed.
- Shared preprocessing module (`preprocess.py`) and mask-to-box conversion (`mask_to_boxes.py`) with unit tests.
- AWS stack: S3, Lambda, SageMaker Serverless Inference, SageMaker Model Registry, EventBridge, DynamoDB, CloudWatch, SNS.
- Python camera & line simulator, Streamlit dashboard.
- Scripted MLOps loop (detect -> collect -> relabel -> retrain -> gate -> approve -> deploy -> verify/rollback), demonstrated once end to end with a nano model.

Out of scope (future work)

- Glass material profile (architecture allows adding it without redesign).
- Physical cameras, factory integration, rotated boxes (OBB).
- Orchestration with SageMaker Pipelines (the scripted workflow is designed to be portable).
- Guaranteed performance on a specific real production line (data is public, not factory-captured).

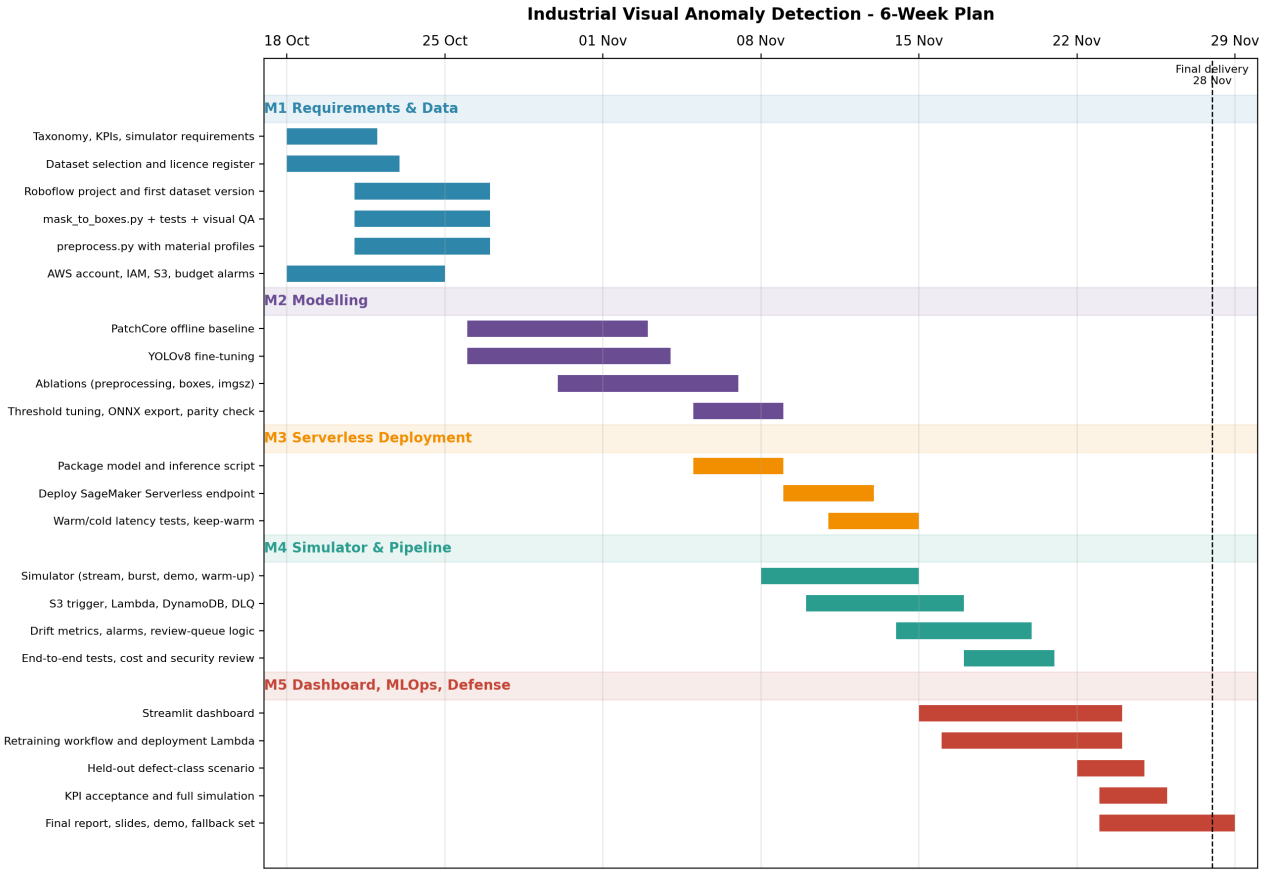
1.2 Project Plan

Timeline

Six weeks, 18 Oct - 28 Nov 2026. The five milestones of the proposal overlap so that the deployment team can start while modelling finishes.

Week	Dates	Focus
1	18 - 24 Oct	M1 starts: requirements, datasets, Roboflow, AWS environment
2	25 - 31 Oct	M1 closes; M2 starts: PatchCore baseline, first YOLOv8 runs
3	1 - 7 Nov	M2: ablations, threshold tuning, ONNX export; M3 starts
4	8 - 14 Nov	M3: endpoint, latency tests; M4 starts: simulator, Lambda
5	15 - 21 Nov	M4 closes: drift metrics, alarms; M5 starts: dashboard, MLOps loop
6	22 - 28 Nov	M5: held-out defect scenario, KPI acceptance, report, defense prep

Gantt Chart



Milestones and Deliverables

Milestone	Window	Objective	Deliverables
M1 Requirements, Data Curation & Environment	18 - 27 Oct	Define requirements; build data and cloud foundations	KPI/requirements specification; dataset register (with conversion parameters); first versioned Roboflow dataset; mask_to_boxes.py and preprocess.py with tests; working AWS environment with budget alarms
M2 Modelling, Ablations & ONNX Benchmarking	26 Oct - 8 Nov	Train and compare models; select the production model	Tracked experiments; ablation and benchmark report; selected ONNX model + thresholds; annotated sample outputs
M3 Serverless Deployment & Latency	5 - 14 Nov	Deploy the model and meet the warm latency target	Model package with version tag; live endpoint; latency report (warm p50/p95/p99, cold start); final accuracy report
M4 Simulator, Pipeline & Monitoring	8 - 20 Nov	Build the simulator and event-driven flow, plus drift monitoring	Simulator with documentation; deployed pipeline (scripts); DynamoDB sample records; drift metrics and alarms; cost estimate and security review
M5 Dashboard, MLOps Loop & Defense	15 - 28 Nov	Complete dashboard, run the retraining loop once, validate, defend	KPI acceptance report; dashboard and deployment guide; retraining and versioning runbook; before/after comparison; final report, presentation, live demo

Resource Allocation

Human resources (6 members, 3 pairs)

Team	Members	Weeks of main effort
Data	Malek Mohamed, Abdelrahman Ramadan	W1-W2 (M1), supports M2 review and M5 relabelling
Modeling	Saeed Ali, Mostafa Mohamed	W2-W3 (M2), supports M3 and M5 retraining
Deployment	Seif Mohamed (Team Lead), Ahmed Khaled	W1 (AWS setup), W3-W6 (M3, M4, M5)

Effort by week (X = main effort, o = support)

Team	W1	W2	W3	W4	W5	W6
Data	X	X	o	o	o	o (relabelling for scenario)
Modeling	o	X	X	o	o	o (retraining run)
Deployment	X	o	X	X	X	X

Technical resources

- Compute: free Colab/Kaggle GPU for training; SageMaker Spot as an alternative.
- AWS: pay-per-request services, Budgets alarms, teardown scripts; keep-warm enabled only for test and defense phases.
- Tools: Python, PyTorch, Ultralytics YOLOv8, Anomalib, OpenCV, ONNX Runtime, Roboflow, Git, Streamlit, Plotly/Altair, boto3.
- Datasets: MVTec AD (non-commercial research licence), NEU, DAGM, one Kaggle/Roboflow ceramics set.

1.3 Task Assignment & Roles

Both members of a pair co-own their team's tasks; tasks are assigned to the pair as a whole. **Project Team Lead: Seif Mohamed.**

Data Team: Malek Mohamed, Abdelrahman Ramadan

Task	Milestone	Output
Select datasets, record name/URL/licence/image count	M1	Dataset register
Create Roboflow project, import and harmonise sources, unify classes	M1	First versioned dataset
Implement and unit-test mask_to_boxes.py; 10% visual QA, 20% second-reviewer check	M1	Tested script, QA log
Implement preprocess.py (geometry, grayscale, ceramic/metal profiles, YAML params)	M1	Tested shared module

Task	Milestone	Output
Duplicate removal, negatives per source, 70/20/10 split by source image group	M1	Clean, leak-free splits
Label review-queue images and publish new dataset version	M5	Dataset for retraining

Modeling Team: Saeed Ali, Mostafa Mohamed

Task	Milestone	Output
PatchCore offline baseline (AUROC, memory size, time)	M2	Baseline report
Fine-tune YOLOv8n/s with augmentation	M2	Trained models, tracked experiments
Ablations: preprocessing variants, box-conversion variants, imgsz 640 vs 960	M2	Ablation report
Per-source/per-material evaluation, mAP by size, image-level recall	M2	Evaluation report
Threshold tuning, ONNX export, parity check, CPU latency	M2	Selected ONNX model + thresholds
Retraining run, evaluation and quality-gate logic	M5	Gated, registered model

Deployment Team: Seif Mohamed (Team Lead), Ahmed Khaled

Task	Milestone	Output
AWS account, IAM least privilege, S3, budget alarms, deployment scripts	M1	Working environment
Package model + inference script, deploy SageMaker Serverless endpoint	M3	Live endpoint
Load tests: warm/cold latency, idle-time-to-cold (5/10/15/30 min), keep-warm	M3	Latency report
Python simulator (stream, burst, demo, --warmup, ground-truth manifest)	M4	Simulator + docs
S3 trigger, inference Lambda, DynamoDB, DLQ, drift metrics, alarms, SNS	M4	Deployed pipeline
Streamlit dashboard (live feed, analytics, performance, model, MLOps panels)	M5	Dashboard
Deployment Lambda: warm-up, smoke test, rollback; Model Registry integration	M5	Gated deployment

Shared Responsibilities (all six)

- KPI acceptance report and full simulation (M5)
- Held-out defect-class scenario run (M5)
- Final report, presentation, demo script and fallback dataset (M5)
- Git hygiene: every dataset, preprocessing and model version linked to a git commit

1.4 Risk Assessment & Mitigation Plan

Likelihood (L) and Impact (I): H = high, M = medium, L = low.

#	Risk	L	I	Mitigation / Fallback
1	Six-week schedule is tight for the full scope	H	H	Milestones overlap by design; drift metrics built in M4; M5 runs one full retraining loop with a nano model; scripted (not SageMaker Pipelines) workflow; cut dashboard panels before cutting KPIs
2	Domain gap between public sources	M	H	Shared preprocessing, negatives from every source, per-source evaluation, material thresholds, documented limitations
3	Tiny defects lost by resizing or noisy boxes	M	H	imgsz 640 vs 960 tested in M2; box-conversion rules with tests and visual QA; mAP by size reported
4	Too few defect samples for some classes	M	M	Combine several sources, augmentation; report limits honestly

#	Risk	L	I	Mitigation / Fallback
5	Cold start during the defense	M	H	EventBridge keep-warm, parallel pre-demo warm-up, Cold/Warming/Warm indicator, recorded fallback, optional Provisioned Concurrency
6	Unseen defect types (YOLOv8 cannot detect them)	M	M	Review queue and retraining loop; stated as a limitation; shown in held-out-class scenario
7	PatchCore memory / cold-start problems	L	L	Not deployed; offline baseline only
8	Train/serve skew	M	H	Single versioned preprocess.py for training and Lambda layer; ONNX parity check
9	Dataset licence issues	L	M	Licence register in M1; non-commercial academic use; cite all sources
10	AWS cost overrun	M	M	Budgets alarms, small model, free Colab/Kaggle GPU or Spot, keep-warm only for test/defense, teardown scripts
11	Delivery slips on integration (model ready late for deployment)	M	H	Deployment team starts with a placeholder ONNX model in W3; interface (input/output format) fixed in M1
12	Team availability / single point of failure	M	M	Each team is a pair with shared ownership; weekly sync; documented runbooks
13	Live system fails during defense	L	H	Fixed demo dataset and pre-recorded results

1.5 KPIs (Key Performance Indicators)

A. System / Model KPIs

KPI	Target	Notes
Image-level recall (critical classes)	>= 95%	Primary KPI; any image with a real defect must FAIL
mAP@0.5 (held-out test set)	>= 0.90	Per material and overall; also by defect size (small/medium/large)
False reject rate	<= 5%	Tunable via per-material confidence threshold
End-to-end latency, warm	< 1.0 s	S3 event to DynamoDB write; p50/p95/p99 in CloudWatch
Cold-start latency	Measured and reported	Separate KPI; mitigated to near-warm on defense day
Audit coverage	100%	Every image/verdict logged with model, dataset and preprocessing versions

B. Reliability / Operations KPIs

KPI	Target	How measured
System uptime during test and defense	target TBD by team (e.g. no failed demo run)	CloudWatch alarms, error rate
Pipeline error rate	Low, with DLQ catching failures	CloudWatch, Lambda DLQ
Throughput	Sustains simulator rate (1-5 images/s steady; burst tested)	Load tests in M3/M4
Drift detection	Alarm + e-mail fires in held-out-class scenario	M5 demo
Rollback	Automatic on failed smoke test	Deployment Lambda test

C. Project-Management KPIs

KPI	Target
Milestone deliverables completed on planned date	5 of 5 (or documented slip with reason)
Unit-test coverage of mask_to_boxes.py and preprocess.py	All five required cases pass (pinhole, fragmented crack, two defects, border defect, empty mask)

KPI	Target
Reproducibility	Every model version traceable to dataset version, preprocessing version and git commit
Cost	Within AWS budget (60 USD)

D. User Adoption

Because the production line and its users are simulated, real adoption cannot be measured. Proxy measures: the dashboard supports a full demo without developer help, and a first-time viewer can read a verdict, a latency panel and the model version (usability check: TBD). Targets are planning values confirmed against the held-out test split and simulated load tests in M3.